چالش ۳۰ روزهٔ اندروید

آمادگی مصاحبهٔ اندروید — جونیور ← مید ← سینیور ← آرکیتکت

نسخهٔ ۲۰۲۶

روز ۹

استراتژی‌های تست، بخش اول — unit test، test doubleها، coroutineها و TDD



اسماعیل مرادی

@esmaeeil-moradi700

چرا تست در ۲۰۲۶ مهم است؟

سؤال‌های تست نشان می‌دهند می‌توانید **بدون ترس کد را تغییر دهید** یا نه.
مصاحبه‌گرها دیگر نمی‌پرسند «تست می‌نویسید؟» — می‌پرسند *چه چیزی* را تست می‌کنید، *کجا*، و با *کدام* test double:

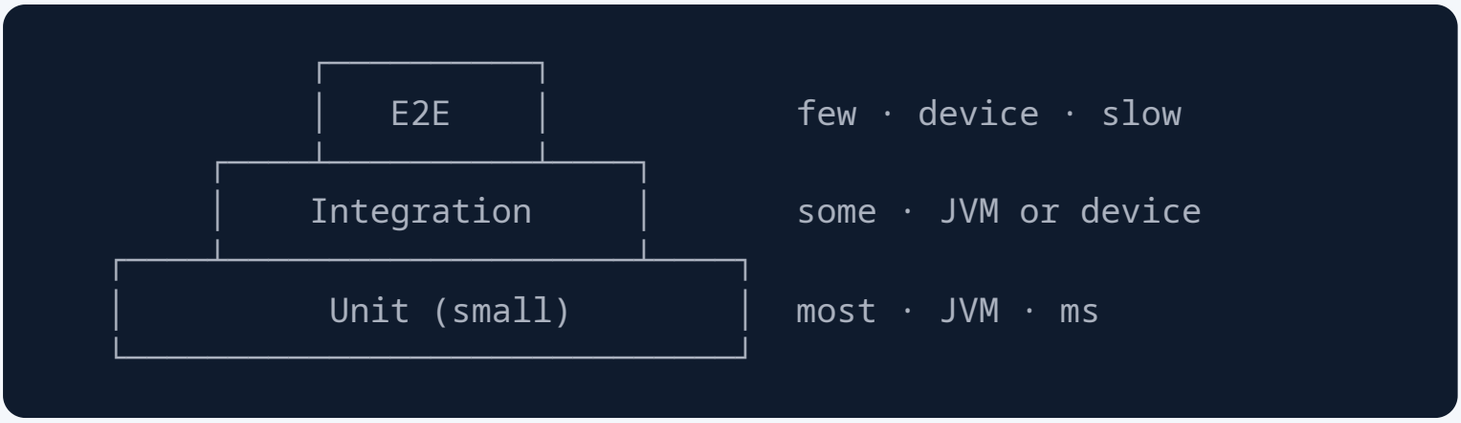
- Compose و UDF صفحه را تابعی از state کردند — بیشتر منطق حالا در **تست‌های سریع JVM** اجرا می‌شود
- Coroutine و Flow به **زمان مجازی** نیاز دارند، نه `Thread.sleep()`
- **fake به‌جای mock** توصیهٔ گوگل است — و انتخاب قابل‌حمل در `commonTest` پروژه‌های KMP، جایی که کتابخانه‌های mock مخصوص JVM اجرا نمی‌شوند
- مجموعهٔ تست کند یا flaky نادیده گرفته می‌شود؛ مجموعهٔ سریع مورد اعتماد است
- بخش دوم (روز ۱۰) تست‌های integration، UI، اسکرین‌شات و performance را پوشش می‌دهد

سطح	سؤال‌ها
جونیور	تست local یا instrumented، آناتومی یک unit test، test doubleها
مید	fake یا mock، کتابخانهٔ MockK، coroutineها با runTest، Flowها با Turbine
سینیور	تست عمیق ViewModel، تست repository، طراحی برای تست‌پذیری
آرکیتکت	استراتژی برای هر لایه، TDD در عمل، مقیاس‌دادن به مجموعهٔ تست



اسماعیل مرادی
@esmaeeil-moradi700

مدل ذهنی: هرم تست



- **دامنهٔ تست** (چه مقدار کد اجرا می‌شود) و **محیط اجرا** (JVM یا دستگاه) دو محور جدا هستند
- پایین‌تر یعنی سریع‌تر، ارزان‌تر و خطاهای دقیق‌تر؛ بالاتر یعنی واقعی‌تر، کندتر و flakyتر
- یک تست فقط وقتی ارزش جایش را دارد که برای باگ واقعی fail شود — و در غیر این صورت pass

قاعدهٔ واحد پشت همهٔ جواب‌های امروز

رفتار را تست کنید، نه پیاده‌سازی را. اگر refactorی که هیچ رفتاری را تغییر نمی‌دهد تست‌هایتان را بشکند، تست‌ها به چیز اشتباهی وابسته‌اند.



اسماعیل مرادی
@esmaeeil-moradi700

جونیور

تست local یا instrumented، آناتومی یک unit test، test doubleها



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱. تست local در برابر تست instrumented

	تست local در `src/test`	تست instrumented در `src/androidTest`
محل اجرا	JVM روی سیستم شما یا CI	دستگاه یا امولاتور
فریم‌ورک اندروید	فایل stub به نام `android.jar` — یا کد واقعی با Robolectric	واقعی
سرعت	چند میلی‌ثانیه برای هر تست	چند ثانیه، به‌علاوهٔ نصب و بوت
کاربرد رایج	منطق، ViewModelها، repositoryها، صفحه‌ها با Robolectric	رفتار دستگاه واقعی، سفرهای E2E، UI سیستم
تسک Gradle	`testDebugUnitTest`	`connectedDebugAndroidTest`

تلهٔ مصاحبه

«*unit test روی JVM اجرا می‌شود و UI test روی دستگاه.*» — این دو محور را قاطی می‌کند. تست یک صفحهٔ Compose می‌تواند با Robolectric روی JVM اجرا شود؛ یک تست کوچک برای کد Keystore به دستگاه نیاز دارد. اول **دامنه** را انتخاب کنید، بعد ارزان‌ترین **محیطی** که دقت کافی دارد.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۲. آناتومی یک unit test خوب

```
class DiscountCalculatorTest {
    private val calculator = DiscountCalculator()

    @Test
    fun `orders over 100 get 10 percent off`() {
        // Arrange
        val order = Order(totalCents = 20_000)
        // Act
        val discount = calculator.discountFor(order)
        // Assert
        assertEquals(2_000, discount)
    }
}
```

- الگوی **Arrange**، **Act** و **Assert** (یا Given، When و Then): یک رفتار در هر تست
- اسم تست، رفتار و نتیجهٔ مورد انتظار را می‌گوید — همان پیام خطاست
- state تازه برای هر تست: JUnit برای هر متد تست یک **نمونهٔ جدید** از کلاس تست می‌سازد



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۲. چیزهایی که مصاحبه‌گر چک می‌کند

- **API عمومی** را تست کنید. helperهای private از طریق همان پوشش داده می‌شوند — نه با `@VisibleForTesting` فقط برای دسترسی به آن‌ها.
- قطعی (deterministic): بدون شبکهٔ واقعی، ساعت، عدد تصادفی یا وابستگی به ترتیب تست‌ها.
- خوانایی مهم‌تر از زرنگی: تست، مستند رفتار است.

تلهٔ مصاحبه

«*فقط یک assert در هر تست.*» — قاعده **یک رفتار** در هر تست است. چند assert که یک نتیجه را توصیف می‌کنند (همهٔ فیلدهای یک خروجی) مشکلی ندارند.

تلهٔ مصاحبه

اسم‌های داخل backtick با فاصله، در تست‌های local روی JVM کار می‌کنند. در تست‌های **instrumented** فقط روی دستگاه‌هایی با API 30 به بالا پشتیبانی می‌شوند؛ برای همین خیلی از تیم‌ها در `androidTest` از `camelCase_underscores` استفاده می‌کنند.

سؤال تکمیلی

«*JUnit 4 یا 5؟*» — تست‌های instrumented روی `AndroidJUnitRunner` اجرا می‌شوند که بر پایهٔ JUnit 4 است. JUnit 5 برای تست‌های local کار می‌کند؛ روی دستگاه فقط با یک پلاگین third-party.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۳. test doubleها — دقیق نام ببرید

نوع	چیست	مثال اندرویدی
Dummy	پاس داده می‌شود ولی هرگز استفاده نمی‌شود	پارامتر `Analytics` که این مسیر کاری با آن ندارد
Stub	جواب ثابت برمی‌گرداند، بدون منطق	`every { flags.isOn(NEW_FEED) } returns true`
Fake	پیاده‌سازی واقعی و سبک	repository درون‌حافظه‌ای، یک `Clock` ثابت
Mock	فراخوانی‌ها را ثبت می‌کند تا تست verify کند	`verify { analytics.log("paid") }`
Spy	شیء واقعی که بعضی فراخوانی‌هایش زیر نظر است یا بازنویسی می‌شود	`spyk(realFormatter)`
Shadow	جایگزین Robolectric برای یک کلاس فریم‌ورک	`ShadowLooper`

تلهٔ مصاحبه

«*در unit test همهٔ وابستگی‌ها باید mock شوند.*» — **نه.** فقط چیزی را جایگزین کنید که کند، غیرقطعی یا خارج از کنترل شماست: شبکه، دیسک، ساعت، عدد تصادفی. value objectها، mapperها و همکارهای خالص **واقعی** می‌مانند.



اسماعیل مرادی
@esmaeeil-moradi700

میدِ

fake یا mock، کتابخانهٔ MockK، coroutineها با runTest، Flowها با Turbine



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۴. fake: رفتار واقعی، بدون فریم‌ورک

```
class FakeUserRepository : UserRepository {
    private val users = MutableStateFlow(mapOf<String, User>())
    var failNextRefresh = false

    override fun observe(id: String): Flow<User?> =
        users.map { it[id] }

    override suspend fun refresh(id: String): Result<Unit> {
        if (failNextRefresh) {
            failNextRefresh = false
            return Result.failure(IOException("offline"))
        }
        return Result.success(Unit)
    }

    fun emit(u: User) = users.update { it + (u.id to u) }
}
```

- state بین فراخوانی‌ها حفظ می‌شود — تست‌ها **نتیجه** را assert می‌کنند، نه ترتیب فراخوانی‌ها را
- یک بار نوشته می‌شود و صدها تست از آن استفاده می‌کنند (در یک ماژول `:core:testing`)



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۴. fake یا mock؟

	Fake	Mock — با MockK یا Mockito
چه چیزی را می‌سنجد	state و نتیجه	تعامل‌ها: فراخوانی، آرگومان، ترتیب
هنگام refactor	به‌ندرت می‌شکند	اغلب می‌شکند — به پیاده‌سازی گره خورده
آماده‌سازی	یک بار نوشته و بازاستفاده می‌شود	`every { }` در هر تست
در پروژهٔ KMP `commonTest`	بله	کتابخانه‌های MockK و Mockito فقط روی JVM

mock وقتی ابزار درستی است که خود تعامل **همان** رفتار باشد: ارسال یک رویداد analytics، یا صدا خوردن یک callback دقیقاً یک بار.

تلهٔ مصاحبه

mock کردن typeهایی که مال شما نیستند — Retrofit، OkHttp، DAOهای Room، `Context` . آن mock حدس شما از رفتارشان را کد می‌کند. آن‌ها را پشت interface خودتان بگذارید، همان را fake کنید، و نسخهٔ واقعی را در یک تست integration بسنجید (روز ۱۰).



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۵. اصول MockK

```
private val api = mockk<UserApi>()
private val analytics = mockk<Analytics>(relaxUnitFun = true)

@Test fun `refresh stores user and logs event`() = runTest {
    coEvery { api.fetchUser("42") } returns UserDto("42", "Sam")
    val dao = FakeUserDao()
    val repo = UserRepository(api, dao, analytics)

    repo.refresh("42")

    assertEquals("Sam", dao.get("42")?.name)
    coVerify(exactly = 1) { api.fetchUser("42") }
    verify { analytics.log("user_refreshed") }
}
```

- `every` و `coEvery` برای stub؛ `verify` و `coVerify` برای چک فراخوانی‌ها (نسخهٔ `co…` برای توابع `suspend`)
- `slot<T>()` همراه با `capture(slot)` برای بررسی آرگومان‌ها؛ `confirmVerified(mock)` برای ممنوع کردن فراخوانی اضافه



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۵. دام‌های MockK

تلهٔ مصاحبه

استفاده از `mockk(relaxed = true)` همه‌جا. فراخوانی‌های stubنشده بی‌صدا `0`، `""` یا لیست خالی برمی‌گردانند — تست pass می‌شود در حالی که کد کار اشتباهی می‌کند. فقط توابع `Unit` را relax کنید (`relaxUnitFun = true`)، آن هم وقتی برایتان مهم نیستند.

تلهٔ مصاحبه

`mockkObject` و `mockkStatic` state سراسری را تغییر می‌دهند. اگر `unmockkAll()` را در `@After` فراموش کنید، mock به تست بعدی نشت می‌کند. نیاز به آن‌ها معمولاً نشانهٔ نبودِ یک seam است (سؤال ۱۰).

سؤال تکمیلی

«Mockito یا MockK؟» — MockK مخصوص Kotlin طراحی شده: stub کردن توابع `suspend`، objectها، توابع top-level و extension. Mockito 5 به‌طور پیش‌فرض از inline mock maker استفاده می‌کند، پس کلاس‌های final کاتلین هم mock می‌شوند؛ `mockito-kotlin` تابع `whenever` را اضافه می‌کند. هر دو فقط روی JVM کار می‌کنند.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۶. coroutineها: `runTest` و زمان مجازی

```
class SyncManager(
    private val api: SyncApi,
    private val io: CoroutineDispatcher = Dispatchers.IO,
) {
    suspend fun syncWithRetry(): Boolean = withContext(io) {
        repeat(3) { attempt ->
            if (api.sync()) return@withContext true
            delay(1_000L * (attempt + 1))
        }
        false
    }
}

@Test fun `gives up after three attempts`() = runTest {
    val api = FakeSyncApi(alwaysFail = true)
    val io = StandardTestDispatcher(testScheduler)
    val sync = SyncManager(api, io)

    assertFalse(sync.syncWithRetry())
    assertEquals(3, api.calls)
    assertEquals(6_000L, currentTime) // virtual ms, instant
}
```



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۶. قواعد `runTest`

- `delay` در **زمان مجازی** جلو می‌پرد؛ `currentTime` نشان می‌دهد ساعت کجاست
- coroutineهای جدید روی `StandardTestDispatcher` **در صف** می‌مانند: `runCurrent()` کارهای همین لحظه را اجرا می‌کند، `advanceTimeBy()` ساعت را جلو می‌برد و `advanceUntilIdle()` همه را تخلیه می‌کند
- **یک scheduler برای همهٔ dispatcherهای تست** — `testScheduler` را پاس بدهید. اگر `Dispatchers.Main` یک `TestDispatcher` باشد، `runTest` از scheduler همان استفاده می‌کند
- `backgroundScope` برای collectorهای بی‌پایان؛ exception گرفته‌نشده در coroutineهای فرزند، تست را fail می‌کند

تلهٔ مصاحبه

«*`runTest` خودش `Dispatchers.Main` را reset می‌کند.*» — نمی‌کند. قبل از هر تست `Dispatchers.setMain()` و بعد از آن `resetMain()` را صدا بزنید — یا از `MainDispatcherRule` روز ۳ استفاده کنید.

تلهٔ مصاحبه

`Dispatchers.IO` هاردکدشده از scheduler تست فرار می‌کند: `delay` در زمان واقعی صبر می‌کند و `advanceUntilIdle()` آن را پوشش نمی‌دهد — نتیجه، تست کند یا flaky است. **dispatcher را inject کنید.**



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۷. Flowها: Turbine

```
// Under test
fun search(q: String): Flow<SearchResult> =
    repo.results(q)
        .map<List<Item>, SearchResult> { SearchResult.Data(it) }
        .onStart { emit(SearchResult.Loading) }

@Test fun `emits loading then results`() = runTest {
    val repo = FakeSearchRepository() // uses a SharedFlow
    val useCase = SearchUseCase(repo)
    val items = listOf(item1)

    useCase.search("kotlin").test {
        assertEquals(SearchResult.Loading, awaitItem())
        repo.emit(items)
        assertEquals(SearchResult.Data(items), awaitItem())
        cancelAndIgnoreRemainingEvents()
    }
}
```

- `awaitItem()`، `awaitComplete()`، `awaitError()`، `expectNoEvents()`
- چند Flow هم‌زمان: `turbineScope { flow.testIn(backgroundScope) }`



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۷. دام‌های تست Flow

تلهٔ مصاحبه

«*تست StateFlow من باید همهٔ stateها را ببیند.*» — `StateFlow` **conflate** می‌کند و مقدارهای مساوی را رد می‌کند. collector *آخرین* مقدار را می‌بیند، پس گذار سریع از `Loading` به `Success` ممکن است فقط `Success` را نشان دهد. برای assert کردن stateهای میانی، زمان‌بندی را کنترل کنید: fakeی که تا اجازهٔ تست suspend می‌ماند، یا یک `StandardTestDispatcher`.

تلهٔ مصاحبه

Flowهای hot به‌طور پیش‌فرض replay ندارند: eventی که یک `SharedFlow(replay = 0)` **قبل از** subscribe شدن `test { }` منتشر کند، از دست رفته است. اول subscribe کنید، بعد عمل را انجام دهید.

- اگر eventها **مصرف‌نشده** بمانند، Turbine تست را fail می‌کند — مصرفشان کنید، یا با `cancelAndIgnoreRemainingEvents()` تمام کنید
- Turbine ندارید؟ در `backgroundScope` و با `UnconfinedTestDispatcher(testScheduler)` داخل یک لیست collect کنید



اسماعیل مرادی
@esmaeeil-moradi700

سینیور

تست عمیق ViewModel، تست repository، طراحی برای تست‌پذیری



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۸. تست ViewModel و stateهای میانی

`ProfileViewModel` در `init` داده را بار می‌کند، state را در یک `MutableStateFlow` نگه می‌دارد و متد `onRetry()` دارد:

```
@get:Rule
val main = MainDispatcherRule(StandardTestDispatcher())

@Test fun `retry after error shows profile`() = runTest {
    val repo = FakeProfileRepository(failFirst = true)
    val handle = SavedStateHandle(mapOf("userId" to "42"))
    val vm = ProfileViewModel(repo, handle)
    assertEquals(ProfileUiState.Loading, vm.uiState.value)

    advanceUntilIdle()
    assertIs<ProfileUiState.Error>(vm.uiState.value)

    vm.onRetry()
    advanceUntilIdle()
    val ready = vm.uiState.value as ProfileUiState.Ready
    assertEquals("42", ready.user.id)
}
```



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۸. انتخاب dispatcher تست

	`UnconfinedTestDispatcher`	`StandardTestDispatcher`
coroutineهای جدید	فوراً شروع می‌شوند	تا وقتی جلو نبرید در صف می‌مانند
مناسب برای	collectorها، تست «state نهایی»	assert کردن `Loading`، ترتیب، زمان‌بندی
ریسک	باگ‌های ترتیب را پنهان می‌کند	اگر جلو بردن را فراموش کنید، هیچ چیز اجرا نمی‌شود

- `SavedStateHandle(mapOf(…))` state **بازیابی‌شده** بعد از process death را شبیه‌سازی می‌کند — بدون نیاز به دستگاه
- stateی که با `stateIn(WhileSubscribed)` ساخته شده به یک collector نیاز دارد (روز ۳) — یا `uiState.test { }`

تلهٔ مصاحبه

با Main از نوع unconfined و fakeی که فوراً جواب می‌دهد، `init` داخل constructor تمام می‌شود: اولین مقداری که می‌خوانید از قبل `Error` است، هرگز `Loading` . این باگ ViewModel نیست — نتیجهٔ dispatcherی است که انتخاب کرده‌اید.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۹. تست repository: قرارداد offline-first

```
@Test fun `failed refresh keeps cached articles`() = runTest {
    val dao = FakeArticleDao(initial = listOf(cachedEntity))
    val api = FakeArticleApi(error = IOException("offline"))
    val repo = ArticleRepository(dao, api)

    val result = repo.refresh()

    assertTrue(result.isFailure)
    assertEquals(
        listOf(cachedEntity.toDomain()),
        repo.observeArticles().first(),
    )
}
```

سناریوها را پوشش دهید، نه خط‌ها را: **cache موجود و موفقیت**، **cache موجود و خطا**، **بدون cache و خطا**، و **تبدیل** DTOها و خطاها.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۹. چه چیزی تست repository را باارزش می‌کند

- **data sourceها** را fake کنید؛ منطق خود repository واقعی بماند
- چیزی را assert کنید که صدازننده می‌بیند: دادهٔ منتشرشده و `Result` برگشتی — نه اینکه کدام متد DAO صدا خورد
- **تبدیل خطا** را تست کنید: از `IOException` به یک خطای domain

تلهٔ مصاحبه

`verify { dao.upsertAll(any()) }` به‌عنوان تنها assert. ثابت می‌کند فراخوانی‌ای رخ داده، نه اینکه داده درست است — و با رفتن به transaction یا متد دیگری از DAO می‌شکند.

تلهٔ مصاحبه

رفتار با cancellation مثل خطا. اگر `refresh()` استثنای `CancellationException` را بگیرد، مثلاً با `runCatching`، آن‌وقت یک refresh لغوشده `Result.failure` برمی‌گرداند و صدازننده برای صفحه‌ای که کاربر ترکش کرده خطا نشان می‌دهد. تستش کنید: API جعلی را suspend نگه دارید، صدازننده را cancel کنید و assert کنید خطایی ثبت نشده.

سؤال تکمیلی

«*DAO جعلی pass می‌شود — آیا SQL درست است؟*» — fake فقط منطق repository را ثابت می‌کند. queryها، استراتژی conflict و migrationها تستی روی Room واقعی می‌خواهند — روز ۱۰.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۰. طراحی برای تست‌پذیری: seam اضافه کنید

سخت برای تست	seam
`Dispatchers.IO` هاردکدشده	یک `CoroutineDispatcher` را inject کنید
`System.currentTimeMillis()` یا `Instant.now()`	`java.time.Clock` را inject کنید؛ در تست `Clock.fixed()`
`UUID.randomUUID()` یا `Random`	یک ID generator یا `Random` با seed ثابت inject کنید
singletonهای `object` که state دارند	یک interface، به‌علاوهٔ DI
`Context` یا resource داخل ViewModel	شناسهٔ resource داخل UI state، یا یک interface کوچک
`android.util.Log` در تست JVM	یک interface برای logger

seam جایی است که تست می‌تواند رفتار را عوض کند **بدون ویرایش کد تحت تست**. تزریق از طریق constructor ساده‌ترین seam است.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۰. ساعت هم یک وابستگی است

```
class TokenStore(
    private val prefs: TokenPrefs,
    private val clock: Clock,
) {
    suspend fun isExpired(): Boolean =
        prefs.expiresAt() <= clock.instant()
}

@Test fun `token is expired exactly at expiry`() = runTest {
    val expiry = Instant.parse("2026-01-01T10:00:00Z")
    val store = TokenStore(
        prefs = FakeTokenPrefs(expiresAt = expiry),
        clock = Clock.fixed(expiry, ZoneOffset.UTC),
    )
    assertTrue(store.isExpired())
}
```

تلهٔ مصاحبه

«*برای فراخوانی زمان از* `mockkStatic` *استفاده کن.*» — کار می‌کند، اما state سراسری را وصله می‌کند و مشکل طراحی را پنهان می‌کند. و `unitTests.isReturnDefaultValues = true` خطاهای `not mocked` را که از `android.jar` می‌آیند خاموش می‌کند — همراه با باگ‌های واقعی.



اسماعیل مرادی
@esmaeeil-moradi700

آرکیتکت

استراتژی برای هر لایه، TDD در عمل، مقیاس‌دادن به مجموعهٔ تست



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۱. استراتژی تست برای هر لایه

چه چیزی	نوع تست	double	محل اجرا
قواعد بیزینسی، mapperها، reducerها	unit	هیچ	JVM
ViewModelها، use caseها	unit	repository جعلی، dispatcher تست	JVM
repositoryها	unit	data source جعلی	JVM
SQL در DAO، JSON، قرارداد API	integration	Room درون‌حافظه‌ای، MockWebServer	JVM یا دستگاه
صفحه‌ها	UI و اسکرین‌شات	state جعلی	JVM با Robolectric، یا دستگاه
سفرهای حیاتی: ورود، پرداخت	E2E	بک‌اند جعلی یا staging	دستگاه

بودجه را جایی خرج کنید که باگ **گران** است و منطق **فشرده** — یک flow پرداخت بیشتر از صفحهٔ تنظیمات تست می‌خواهد.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۱. دام‌های هرم تست

تلهٔ مصاحبه

«*unit testها به ۷۰ تا ۹۰ درصد coverage نیاز دارند و integration به ۱۰ تا ۲۰ درصد.*» — این دو ایده را قاطی می‌کند. نسبت معروف **۷۰/۲۰/۱۰** راهنمایی است برای اینکه از هر **نوع** تست چه تعداد بنویسید. coverage یعنی چه مقدار از کد را تست‌ها اجرا می‌کنند — یک معیار دیگر.

تلهٔ مصاحبه

«*coverage صددرصد یعنی اپ تست شده.*» — coverage نشان می‌دهد چه چیزی **اجرا شد**، نه چه چیزی **بررسی شد**. تستی بدون هیچ assert هم coverage را بالا می‌برد.

سؤال تکمیلی

«*هرم یا testing trophy؟*» — شکل کمتر از اصل مهم است: بیشترین اطمینان به ازای هر ثانیه زمان تست. با Robolectric و تست اسکرین‌شات، خیلی از تست‌های «UI» حالا روی JVM اجرا می‌شوند، پس وسط هرم ارزان‌تر شده است.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۲. TDD در عمل

```
// 1. RED — reproduce the bug report as a test
@Test fun `iban with spaces is valid`() {
    val iban = "DE89 3704 0044 0532 0130 00"
    assertTrue(IbanValidator.isValid(iban))
}

// 2. GREEN — the smallest change that passes
fun isValid(raw: String): Boolean {
    val iban = raw.replace(" ", "")
    return iban.length in 15..34 && checksumOk(iban)
}

// 3. REFACTOR — clean up; the test is your safety net
```

TDD خوب جواب می‌دهد	TDD خوب جواب نمی‌دهد
منطق خالص، parserها، reducerها، validatorها	spikeهای اکتشافی، APIهای ناشناخته
هر رفع باگ — اول تست، بعد رفع	چیدمان بصری (به‌جایش تست اسکرین‌شات)

تلهٔ مصاحبه

«*TDD یعنی همهٔ تست‌ها را از اول بنویسی.*» — TDD یک حلقهٔ کوتاه است: **یک** تست fail، pass کردنش، refactor، و تکرار.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۳. مقیاس‌دادن به مجموعهٔ تست: fakeهای قابل‌اعتماد

```
abstract class UserRepositoryContract {
    abstract fun createRepository(): UserRepository

    @Test fun `saved user can be read back`() = runTest {
        val repo = createRepository()
        repo.save(user42)
        assertEquals(user42, repo.observe("42").first())
    }
}

// src/test — the fake must honour the same contract
class FakeUserRepositoryTest : UserRepositoryContract() {
    override fun createRepository() = FakeUserRepository()
}
// Robolectric or androidTest — same tests, real Room impl
```

- fakeهای مشترک در یک ماژول `:core:testing` زندگی می‌کنند — یک fake برای هر interface، نه یکی برای هر تست
- **contract test** همان مجموعهٔ تست را روی fake و پیاده‌سازی واقعی اجرا می‌کند تا fakeها از واقعیت فاصله نگیرند



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۳. سنجش سلامت تست‌ها

سیگنال	چه می‌گوید	مراقب باشید
coverage خط و شاخه (JaCoCo ،Kover)	کدی که هیچ تستی اجرایش نمی‌کند	coverage بالا لزوماً یعنی assert خوب نیست
امتیاز mutation (مثلاً PIT)	اینکه تست‌ها باگ تزریق‌شده را می‌گیرند یا نه	کند است — روی ماژول‌های هسته اجرا کنید
مدت اجرای مجموعه	سرعت بازخورد	مجموعهٔ کند نادیده گرفته می‌شود
نرخ flaky	اعتماد به build قرمز	روز ۱۰
باگ‌هایی که به production رسیدند	چیزی که تست‌ها از دست دادند	هر کدام یک تست جدید می‌شود

جواب سینیور در یک جمله

«رفتار را در پایین‌ترین سطحی تست کن که می‌تواند باگ را بگیرد، با fakeهایی که می‌شود به آن‌ها اعتماد کرد، و مجموعه را آن‌قدر سریع نگه دار که هیچ‌کس از اجرایش نگذرد.»



اسماعیل مرادی
@esmaeeil-moradi700

روز ۹ — خلاصهٔ یک‌صفحه‌ای

- دامنهٔ تست (unit، integration، E2E) و محیط اجرا (JVM یا دستگاه) **دو محور جدا** هستند.
- یک رفتار در هر تست، الگوی Arrange، Act و Assert، و اسم‌هایی که مثل پیام خطا خوانده شوند.
- فقط وابستگی‌های کند یا غیرقطعی را جایگزین کنید — **fake به‌جای mock**.
- mock وقتی که خود تعامل، رفتار است؛ هرگز typeهایی را که مال شما نیستند mock نکنید.
- `runTest` یعنی زمان مجازی؛ یک scheduler برای همهٔ dispatcherهای تست؛ و Main را **reset نمی‌کند**.
- dispatcher، ساعت و ID generator را inject کنید — seam بهتر از `mockkStatic` است.
- `StateFlow` conflate می‌کند: برای دیدن `Loading` زمان‌بندی را کنترل کنید؛ برای دنبالهٔ مقدارهای Flow از Turbine.
- تست repository قرارداد offline-first را می‌سنجد و هرگز cancellation را به خطا تبدیل نمی‌کند.
- نسبت ۷۰/۲۰/۱۰ دربارهٔ **نوع** تست است، نه coverage؛ و coverage مساوی کیفیت نیست.
- contract testها fakeهای مشترک را صادق نگه می‌دارند.

روز ۱۰ در راه است: استراتژی‌های تست، بخش دوم — تست integration، UI و اسکرین‌شات. این پست را ذخیره کنید و همراه ما باشید.



اسماعیل مرادی
@esmaeeil-moradi700